

PROTOCOL

STRK20 VS PROTOCOL 01

DOCUMENT TECHNIQUE · DIFF EXACT

Mécanique complète du framework STRK20, établie sur le code source ouvert [starkware-libs/starknet-privacy](https://github.com/starkware-libs/starknet-privacy), et comparaison axe par axe avec la stack Protocol 01, établie sur le dépôt.

12

AXES DE DIFF

3

CONSTATS BLOQUANTS

9

CORRECTIONS FICHE

2

CIBLES PQ

RÈGLE DE LECTURE

Chaque affirmation porte sa source : fichier Cairo / Rust / TypeScript pour STRK20, file:line du dépôt pour Protocol 01. Là où le code contredit le brief du 22 juillet, c'est signalé en clair.

DOCUMENT TECHNIQUE · BUILDATHON STRK20 PARIS

22 juillet 2026

Volta Team · Developed by Slashy Fx

SOMMAIRE

SECTION 0 · P.3

À lire en premier

Trois constats qui changent le plan de la journée.

SECTION 1 · P.4-11

Comment STRK20 marche

- Modèle : pas de Merkle, pas de commitment
- Chiffrement : ECDH courbe STARK, hash-and-add
- Adressage du destinataire, discovery
- Transaction en 8 phases
- Où la preuve est vérifiée (SNIP-36)
- Stwo, Circle STARK, AIR de CPU fixe
- Compliance et constats de confiance
- Surface développeur

SECTION 2 · P.12-15

Protocol 01, vérité terrain

- Corrections à la fiche technique
- Les 7 circuits, table exacte
- Compute units : ce qu'on peut affirmer
- Pool, Specter, et faiblesses connues

SECTION 3 · P.16-20

Le diff, axe par axe

1. Système de preuve et corps
2. Modèle d'écriture des contraintes
3. Lieu et coût de vérification
4. Transport de la preuve
5. Modèle de note
6. Accumulateur
7. Nullifiers
8. Découverte du destinataire
9. Chiffrement des notes
10. Viewing keys et compliance
11. Modèle de compte
12. Composabilité et maturité

SECTION 4-5 · P.21-22

Le trou PQ & le branchement

Formulation exacte à employer, les deux cibles concrètes, et le point de branchement Specter révisé.

SECTION 6-8 · P.23-24

Questions, retraits, sources

Questions révisées à poser sur place, formulations à retirer avant de parler, et l'appareil de sources avec ses réserves.

Provenance

STRK20 a été ouvert en Apache 2.0 le 8 juillet 2026. Tout ce qui le concerne ici vient du dépôt [starkware-libs/starknet-privacy](#) (contrat Cairo, service de discovery Rust, SDK TypeScript, preuves Lean 4), de SNIP-36, des types [@starknet-io/types-js@0.10.3](#), et de la roadmap post-quantique StarkWare du 30 juin 2026. Aucune affirmation ne repose sur les pages marketing.



Trois constats issus du code, chacun invalidant une hypothèse de la roadmap du buildathon.

CONSTAT 1

STRK20 est open source

github.com/starkware-libs/starknet-privacy — Apache 2.0, ouvert le 8 juillet 2026. Contient le contrat Cairo du pool, le service de discovery en Rust, le SDK TypeScript, les preuves formelles Lean 4 et les contrats anonymizer. Pool mainnet déployé à 0x040337b1af3c663e86e333bab5a4b28da8d4652a15a69bbee2b677776ffe812a.

Le risque « SDK gated » du plan initial n'existe plus. Ce qui est réellement gated : l'URL du **proving service**, celle de l'**indexer** (variables d'environnement sans valeur par défaut publiée nulle part), l'**adresse du pool Sepolia**, et le paquet npm qui vit sur GitHub Packages, pas sur npm public.

CONSTAT 2

Le projet prévu ne peut pas marcher

Le plan était : Specter dérive une adresse furtive, STRK20 transfère en privé vers elle. **Impossible**. Il n'existe aucune adresse furtive dans STRK20, et le destinataire doit être un compte Starknet déployé **et enregistré** avec une clé de vue publique on-chain.

```
// open_channel
let sender_public_key = self.public_key.read(sender_addr);
assert(sender_public_key.is_non_zero(), errors::SENDER_NOT_REGISTERED);
let recipient_public_key = self.public_key.read(recipient_addr);
assert(recipient_public_key.is_non_zero(), errors::RECIPIENT_NOT_REGISTERED);
```

packages/privacy/src/privacy.cairo:368-379

Leur contournement documenté est un contrat d'escrow avec lien de claim off-chain, pas de la découverte de destinataire. Le point de branchement doit bouger — voir section 5.

CONSTAT 3

Le vrai trou est meilleur que celui prévu

À l'enregistrement, le contrat chiffre lui-même la clé de vue **privée** de l'utilisateur vers la clé publique de l'auditeur, et la stocke on-chain de façon immuable.

```
pub(crate) struct EncPrivateKey {
    pub auditor_public_key: felt252,
    pub ephemeral_pubkey: felt252,
    /// `enc_private_key = h(ENC_PRIVATE_KEY_TAG, rK.x) + private_key`
    pub enc_private_key: felt252,
}
```

packages/privacy/src/objects.cairo:42-53

Une clé qui déchiffre l'historique bidirectionnel complet d'un utilisateur, chiffrée par ECDH sur la courbe STARK, publiée on-chain, immuable (WriteOnce), permanente. C'est la surface harvest-now-decrypt-later la plus dense du système. La Q3 du brief visait juste ; le code la confirme mot pour mot.

La surprise architecturale principale : elle inverse l'intuition de quiconque vient de Tornado, Zcash, ou de tes propres pools dénommés.

Il n'y a pas d'arbre de Merkle

Pas d'accumulateur, pas de racine, pas de fenêtre de racines historiques. Un `grep -ril merkle` sur tout le dépôt ne renvoie qu'un seul fichier, et le hit parle du trie d'état de Juno, pas du protocole. Les notes vivent directement dans des maps de stockage Cairo.

```
notes: Map<felt252, Note>,
nullifiers: Map<felt252, bool>,
recipient_channels: Map<ContractAddress, Vec<EncChannelInfo>>,
```

packages/privacy/src/privacy.cairo:88-99

Une note n'est pas un hash-commitment

```
pub struct Note {
    /// The packed value of the note `(salt, amount)`
    pub packed_value: felt252,
    /// The token address of the note (zero for encrypted notes).
    pub token: ContractAddress,
}
```

packages/privacy/src/objects.cairo:89-100

Le propriétaire, le token et l'index ne sont pas dans un préimage de commitment : ils sont liés **implicitement par la clé de stockage**.

```
note_id = Poseidon(NOTE_ID_TAG, channel_key, token, index, 0)
nullifier = Poseidon(NULLIFIER_TAG, channel_key, token, index, 0, owner_private_key)
```

packages/privacy/src/ hashes.cairo:200-236

La conséquence à retenir

Comme `channel_key` est un secret partagé uniquement entre l'émetteur et le destinataire, les slots de stockage sont pseudo-aléatoires pour tout observateur. **Le pseudo-aléatoire de l'adresse de stockage remplace l'anonymat d'ensemble d'un arbre de Merkle**. C'est une hypothèse de sécurité différente, et c'est exactement là que l'argument post-quantique mord le plus fort.

Dépenser : lecture de stockage, pas preuve d'appartenance

```
let subchannel_marker = compute_subchannel_marker(...);
assert(self.subchannel_exists.read(subchannel_marker), errors::SUBCHANNEL_NOT_FOUND);
let packed_value = self.notes.entry(note_id).packed_value.read();
assert(packed_value.is_non_zero(), errors::NOTE_NOT_FOUND);
```

packages/privacy/src/privacy.cairo:585-629

Asymétrie du nullifier — un point où ils sont devant

Le nullifier contient la clé privée du **propriétaire**. L'émetteur d'une note ne peut donc pas calculer son nullifier, et ne peut pas observer quand son paiement est dépensé. Protocol 01 n'a pas cette propriété.

Pas de KEM au sens moderne, pas de chiffrement symétrique authentifié. Deux couches : ECDH sur la courbe STARK, puis masquage additif.

Couche 1 – ouverture de canal (asymétrique)

ECDH sur la courbe STARK avec un scalaire éphémère r par canal. Seule l'abscisse du point est utilisée. La KDF est un simple Poseidon domain-séparé, et le « chiffrement » est une addition dans le corps de base.

```
fn _compute_shared_x(ephemeral_secret: felt252, public_key: felt252) → (felt252, felt252) {
    let ephemeral_pub_point = GEN_P().mul(scalar: ephemeral_secret);
    let public_point = EcPointTrait::new_from_x(x: public_key)...;
    let shared_point = public_point.mul(scalar: ephemeral_secret);
    let shared_x = ...x();
}
```

packages/privacy/src/utils.cairo:123-145

```
enc_channel_key = h(CHANNEL_KEY_TAG, rK.x) + channel_key
enc_sender_addr = h(SENDER_ADDR_TAG, rK.x) + sender_addr
```

Couche 2 – données dans le canal (symétrique)

```
enc_amount = (Poseidon(ENC_AMOUNT_TAG, channel_key, token, index, 0, salt) + amount) mod 2^128
enc_token = Poseidon(ENC_TOKEN_TAG, channel_key, index, 0, salt) + token
packed_value = salt * 2^128 + enc_amount // salt 120 bits, choisi par l'émetteur
```

packages/privacy/src/ hashes.cairo:212-222

Dérivation de la clé de canal

```
channel_key = Poseidon(CHANNEL_KEY_TAG, sender_addr, sender_private_key,
                      recipient_addr, recipient_public_key)
```

Les canaux sont strictement **directionnels** : Alice→Bob et Bob→Alice sont deux canaux distincts. Un dépôt est un canal de soi vers soi.

Contrainte de clé canonique

Comme seules les abscisses servent à l'ECDH, k et $ORDER-k$ seraient indistinguables. Le contrat impose donc $is_canonical_key(key) = key < HALF_ORDER$. Une clé hors borne compile mais dérive silencieusement de mauvaises clés de canal : les notes ne se déchiffrent jamais.

Le schéma est figé dans le contrat

Aucun champ de version ou d'algorithme sur Note ni sur EncChannelInfo. Le seul versionnage est le suffixe :V1 dans les 17 tags Poseidon. Changer de schéma exige un upgrade de contrat via ReplaceabilityComponent. **Réponse à la Q2 : non, pas extensible.**

Audit OpenZeppelin — le r éphémère

Le scalaire éphémère est fourni par le client et seulement validé non nul. Le contrat « n'impose pas l'unicité ni n'empêche la réutilisation de r », d'où un risque de réutilisation déterministe de ciphertext. Second constat : une `auditor_public_key` invalide déclenche des panics `EcPointTrait::new_from_x(...).unwrap()` capables de bloquer enregistrements et retraits à l'échelle du système. Commit audité c5e2fb53...

Réponse à la question n°1 du brief. Elle est nette, et elle est structurante.

Il n'y a **ni tag de détection, ni view tag, ni scan de tout le pool**. Le contrat stocke un seau par destinataire, indexé par son adresse Starknet **en clair**.

```
/// Map of recipient_addr to a list of their encrypted channels.
recipient_channels: Map<ContractAddress, Vec<EncChannelInfo>>,
```

packages/privacy/src/privacy.cairo:88-89

L'émetteur ajoute un EncChannelInfo dans le seau du destinataire via l'action serveur Append. Le destinataire ne lit que son propre seau et déchiffre par essai chaque entrée.

La fuite qui en découle

Le slot de stockage est dérivé déterministiquement de l'adresse en clair via `get_storage_var_address("recipient_channels", [recipient_address])`. Tout observateur des state diffs voit donc qu'une adresse précise vient de recevoir un nouveau canal.

Seul le **premier** transfert d'une paire (émetteur, destinataire) fait cela ; les notes suivantes vont dans des slots dérivés du `channel_key` secret. Le SDK émet d'ailleurs un warning `WarningCode.USER_LINKAGE` quand plusieurs `OpenChannel` apparaissent dans une même transaction. Leurs propres docs listent la « channel-open linkability » comme limite connue.

La discovery : balayage d'index dense à 3 niveaux

NIVEAU	OPÉRATION
1	Lire son propre <code>recipient_channels</code> et déchiffrer par essai ECDH chaque entrée
2	Parcourir les index de sous-canal 0, 1, 2... par canal, en démasquant le token
3	Parcourir les index de note 0, 1, 2... par (canal, token), démasquer les montants, tester chaque nullifier dérivé contre l'état on-chain

Le balayage s'arrête au premier slot vide. Tous les slots sont `WriteOnce` et les index doivent être séquentiels (`INDEX_NOT_SEQUENTIAL`) : c'est la densité qui garantit la terminaison.

Le point où ils sont architecturalement devant

Le coût de scan est proportionnel à ton activité, pas au volume du pool. « A pool with millions of transfers costs you no more to scan than a quiet one. » La discovery est en outre une lecture gratuite : pas de preuve, pas de frais, pas de soumission. Les notes ne sont dépensables que 10 blocs après création.

Ce qui reste public à chaque bord

L'événement `Deposit` porte `user_addr`, `token`, `amount` en clair. `Withdrawal` porte `to_addr`, `token`, `amount` en clair plus un `enc_user_addr` chiffré pour l'auditeur. `ViewingKeySet` lie publiquement une adresse à sa clé de vue publique. Leurs docs le résument ainsi : « edges are public by design ».

Pas de triplet shield / transfer / unshield. Une liste d'actions groupées en 8 phases strictement ordonnées.

PHASE	ACTION	RÔLE
0	SetViewingKey	Enregistrement. Immuable une fois posé (WriteOnce)
1	OpenChannel	Crée le canal directionnel émetteur → destinataire
2	OpenSubchannel	Un sous-canal par token dans le canal
3	Deposit	Entrée de fonds publics dans le pool. Toujours vers soi
4	UseNote	Consomme une note, écrit son nullifier
5	CreateEncNote / CreateOpenNote	Crée les notes de sortie
6	Withdraw	Sortie de fonds vers une adresse publique
7	InvokeExternal	Appel d'un contrat anonymizer. Au plus une fois par tx

packages/privacy/src/actions.cairo:275-316 – assert(action_phase ≥ curr_phase, errors::ACTIONS_OUT_OF_ORDER)

Traduction depuis ton vocabulaire

SHIELD

Deposit + CreateEncNote vers soi

TRANSFER

UseNote +
CreateEncNote(destinataire) +
CreateEncNote(monnaie rendue)

UNSHIELD

UseNote + Withdraw

La conservation de la valeur n'est pas une contrainte de circuit

C'est un registre de balance temporaire par token, à l'intérieur d'une transaction. Deposit et UseNote ajoutent, CreateNote et Withdraw retranchent. La balance ne peut jamais devenir négative, et chaque token doit finir exactement à zéro.

```
fn subtract_balance(...) {
    let new_value = current_balance.checked_sub(amount)
    .expect(errors::NEGATIVE_INTERMEDIATE_BALANCE);
}
fn assert_valid(self: SquashedTokenBalances) {
    for (_token, _, balance) in self.into_entries() {
        assert(balance.is_zero(), errors::FINAL_BALANCE_MUST_BE_ZERO);
    }
}
```

packages/privacy/src/objects.cairo:9-27

Point de comparaison direct : eux résolvent la conservation par une machine à états on-chain, toi par la dénomination du pool. **Ni l'un ni l'autre ne le met dans une AIR.** C'est un bon terrain de discussion entre pairs.

Notes ouvertes — l'échappatoire DeFi assumée

salt = OPEN_NOTE_SALT (=1) : le token et le montant sont en clair, et le montant est écrit après génération de la preuve par un contrat anonymizer. Les notes chiffrées ont toujours salt ≥ 2 ; salt = 0 signifie que la note n'existe pas. « Open notes trade amount privacy for that late binding. »

Le chemin exact, souvent mal décrit. C'est l'axe où l'écart architectural avec Protocol 01 est le plus grand.

#	ÉTAPE
1	Le SDK compile les actions et les envoie à un proving service opéré par un tiers
2	Le service exécute dans un bloc Starknet virtuel (Virtual SNOS), un OS modifié à fonctionnalité réduite. Les builtins <code>add_mod</code> , <code>mul_mod</code> et Keccak sont exclus des claims prouvés en phase 1
3	La preuve Stwo voyage dans les champs optionnels <code>proof</code> (<code>uint32[]</code>) et <code>proof_facts</code> (<code>felt252[]</code>) d'une Invoke V3 — pas dans le calldata
4	Un vérificateur S-two en Rust embarqué dans le gateway / séquenceur vérifie. Pas un contrat Cairo. Pas de récursion dans la preuve d'état L2 en phase 1
5	Le contrat de pool lit seulement les petits <code>proof_facts</code> via un syscall dédié, et valide la variante de programme, le bloc de base, la fenêtre de validité et le hash de message

SNIP-36 — community.starknet.io/t/snip-36-in-protocol-proof-verification/116123

Deux aveux qu'ils écrivent noir sur blanc

« *applications built with this feature will have **degraded security** compared to native Starknet applications* »
— à corriger dans une release ultérieure.

« *The protocol is **not zero-knowledge by default** ; proofs reveal no execution data beyond declared public input.* »

En phase 1 les preuves sont vérifiées par le consensus mais pas par le SNOS, ne sont pas dans les blocs, ne sont pas récupérables au feeder gateway, et ne sont conservées que quelques semaines. Les full nodes ne re-vérifient pas. La confidentialité v0.14.2 est décrite comme « de facto », le vrai ZK est reporté en phase 2.

Pourquoi SNIP-36 existe — et pourquoi c'est ton meilleur angle

« *Including the full proof (often tens of thousands of felts) in calldata exceeds the present transaction limit (5K felts).* »

La vérification dans un contrat Cairo a bien été tentée — Integrity, de Herodotus, déployé sur Starknet — et a dû être découpée en plusieurs contrats et plusieurs transactions à cause des limites de classhash, de calldata et de pas Cairo (3M pas par transaction). Autrement dit : **ils ont rencontré exactement ton problème, et l'ont résolu en changeant le protocole de la L2 plutôt qu'en optimisant le vérificateur**. Cette option ne t'est pas offerte sur Solana.

Fenêtre de validité de preuve — leur analogue de tes racines historiques

Le contrat rejette les preuves plus vieilles que `proof_validity_blocks` (450 blocs, ~15 min à 2 s/bloc). Convention du SDK : prouver à `currentBlock - 10`. Tout état on-chain lu par la preuve (clé de vue du compte, balance du déposant, set de nullifiers) doit avoir été écrit au moins 10 blocs avant le bloc de base. Ce n'est pas le même mécanisme que le tien : le tien résout la concurrence sur l'état de l'arbre, le leur la fraîcheur de l'exécution virtuelle.

Stwo, Circle STARK sur M31. Et le point architectural qui explique tout le reste : Cairo est une AIR de CPU fixe.

Le corps : M31, $p = 2^{31} - 1$

Tour M31 → CM31 (ext. quadratique) → QM31 (ext. quartique, utilisée pour Fiat-Shamir et les aléas DEEP/FRI).

Pourquoi pas un corps 64 bits

M31 a une 2-adicité de 1 ($p-1 = 2 \cdot (2^{30}-1)$) : aucun sous-groupe lisse en 2^k pour une FFT radix-2.

Le Circle STARK contourne ça via la courbe circulaire $x^2 + y^2 = 1$, dont le groupe de points a un ordre de $p+1 = 2^{31}$, exactement une puissance de deux. La CFFT garde la même structure de papillon radix-2, sur des polynômes bivariés restreints au cercle. Papier : Haböck, Levit, Papini, ePrint 2024/278, annonçant 1,4× de gain sur un STARK BabyBear.

Le vrai argument est la densité SIMD : la réduction mod $2^{31}-1$ est un décalage cyclique, pas une division, et l'arithmétique tient dans des lanes 32 bits AVX2 / AVX-512 / NEON / WASM-SIMD.

Hash de commitment	Blake2s-256 par défaut ; Poseidon252 en option pour règlement Ethereum
Arbre de Merkle	« lifté » : le hash d'un nœud dépend des enfants et des colonnes injectées à cette couche — colonnes de hauteurs différentes sans padding
Paramètres par défaut	log_blowup = 1 (blowup 2), 70 requêtes, 26 bits de grinding, plus un PoW de phase d'interaction
Soundness cible	96 bits conjecturés (régime list-decoding), pas prouvés sous borne de Johnson
Trusted setup	aucun — « fully transparent, no toxic waste, no MPC ceremony »

Le point architectural décisif

Cairo est **une AIR de CPU fixe**. Un unique jeu immuable de contraintes polynomiales prouve l'exécution valide de la machine Cairo ; le programme applicatif n'est pas compilé en contraintes, il est **de la donnée** placée en mémoire publique et consommée comme trace.

« We describe a single set of polynomial equations for the statement that the execution of a program on this architecture is valid. Given a statement one wishes to prove, Cairo allows writing a program that describes that statement, instead of writing a set of polynomial equations. » — Cairo paper, ePrint 2021/1063

En pratique stwo-cairo décompose ça en 69+ composants (un par opcode, un par builtin) reliés par des arguments de lookup LogUp. Les builtins sont des périphériques mappés en mémoire qui remplacent un calcul coûteux dans la VM par un composant d'AIR dédié. C'est ça, « le circuit et le contrat sont le même code » : pas de DSL de circuit, pas d'AIR par application, pas de vérificateur déployé par application.

Benchmark officiel S-two — 48 cœurs EPYC, CPU seul			
CHARGE	PREUVE	TAILLE	VÉRIF.
Fibonacci $n = 2^{20}$	18,61 s	808 KB	414 ms
SHA-256 $n = 2^{18}$ octets	121 s	1315 KB	83 ms

L'axe où l'écart avec Protocol 01 est maximal, dans les deux sens : ils ont une couche complète, et c'est leur surface la plus exposée.

Clé d'auditeur	une seule <code>auditor_public_key</code> : felt252 en stockage, posée au déploiement, modifiable par le rôle <code>security_governor</code> . Les docs parlent de clés à seuil ; le contrat n'en a pas trace
Chiffrement de la clé	ECDH éphémère sur courbe STARK + masque Poseidon, on-chain, immuable (<code>WriteOnce</code>), avec un événement public <code>ViewingKeySet</code>
Portée de divulgation	historique bidirectionnel complet : canaux entrants et sortants, chaque montant, appariement nullifier→note, traçage avant et arrière
Pouvoir de l'auditeur	voir, jamais dépenser — la dépense exige une signature de compte vérifiée dans la preuve
Vérification formelle	~60 fichiers Lean 4 : notes, canaux, sous-canaux, no-replay, spendability, compliance, traçage

Screening de dépôt appliqué on-chain

Depuis v0.14.3, chaque dépôt porte une attestation SNIP-12 `DepositorValidation { depositor, issued_at }` signée par un screener off-chain dont la clé publique STARK est en stockage. Le pool vérifie l'ECDSA. Expiration 300 s, tolérance de dérive future 60 s.

Faire tourner son propre prover ne contourne pas ça. Divergence non expliquée : les docs nomment « FPI » comme screener, le code appelle l'API d'Elliptic via un proxy GCP `elliptic-proxy`. Avec en plus des listes opérateur `blockOverrideAddresses` et `additionalBlockedAddresses`.

Deux constats de confiance

Ce ne sont pas des accusations : c'est ce que disent leur code et leurs specs. En phase 1 ce sont des choix d'architecture explicites.

La clé de vue privée part en clair au proving service

```
const executeViewCalldata = callDataCompiler.compile("compile_actions", [
  userAddress, user.viewingKey, cairoActions
]);
```

`sdk/src/internal/proof-invocation-factory.ts:128-140`

Un prover hébergé apprend donc la clé qui déchiffre l'historique entier de l'utilisateur.

Le discovery service reçoit des clés et renvoie du déchiffré

« The service decrypts and returns decrypted channels and unspent notes. Decryption keys must be provided per request. »

« The operator can observe : which recipients are active and when. How many channels, subchannels, and notes each recipient has. Token addresses used per channel. Timing of sync activity. »

`crates/discovery-service/specs/05-security-considerations.md`

La mitigation est OHTTP (basé X25519, relais optionnel), et elle ne couvre que la corrélation d'IP.

Privacy Wallet API = Starknet Wallet API v0.10.3, publiée en types TypeScript. Exactement trois méthodes. Aucune ne s'appelle shield, unshield ou claim.

```
wallet_strk20InvokeTransaction // { actions: STRK20_ACTION[] } → { transaction_hash }
wallet_strk20PrepareInvoke    // { actions, simulate? }      → STRK20_CALL_AND_PROOF
wallet_strk20Balances         // { tokens: Address[] }      → STRK20_BALANCE_ENTRY[]
```

@starknet-io/types-js@0.10.3 — dist/types/wallet-api/methods.d.ts:135,157,174

Les 4 variantes d'action

deposit (toujours vers soi, pas de champ recipient),
withdraw, **transfer** (accepte le littéral "OPEN" comme montant), **invoke**.

Pas de scan de notes dans la Wallet API

Une dapp ne lit que des balances agrégées par token. Le scan n'existe que dans le SDK bas niveau (discoverNotes, discoverChannels), qui exige la clé de vue. Délibéré : « A normal dapp should not receive the user's viewing key. »

Le SDK bas niveau — builder fluide

```
await transfers.build(opts)
  .with(TOKEN, t => t.inputs(note).transfer({ recipient, amount }).withdraw({ amount }))
  .surplusTo(self)
  .execute({ provingBlockId });
```

Verbes : register, deposit, transfer, withdraw, setup, invoke.

Trois pièges de soumission qui mordront toute intégration

- proofFacts: [] doit être **omis entièrement** (spread conditionnel), sinon starknet.js sérialise une v3 invalide
- tip: 0n est obligatoire, sinon Cannot mix BigInt and other types
- Appeler invalidateProofNonceCache() après tout échec de soumission, sinon INVALID_NONCE

Côté dapp

starknet.js ≥ 10.4.0 et la classe WalletAccountV6. Attention : le tag npm latest est 10.0.2, les versions capables STRK20 sont sur next (10.5.1). À épingler

Où tourne le prover

côté serveur, ni dans le wallet ni dans la dapp. Auto-hébergement possible via image container, mais c'est une charge ops et StarkWare dit les proving services auto-hébergés en phase 1

Temps de preuve

chiffres contradictoires non réconciliés : ~29 s (12 cœurs / 46 GiB) dans le diagramme de pipeline, ~4 s dans le README du SDK, « few-seconds area » selon l'auteur de SNIP-36

Wallets

Ready live côté dapp ; Xverse a le privacy in-wallet mais son support Wallet API côté dapp est en cours. Détecter les capacités à l'exécution

Frais

fixe par transaction, 4 STRK annoncés — « closer to a gas fee than a toll »

Composabilité DeFi — les contrats anonymizer

Un helper expose une entrée obligatoire unique privacy_invoke, qui assure que l'appelant est le pool. Le pool retire les tokens vers le helper, l'appelle, et le helper renvoie des instructions OpenNoteDeposit { note_id, token, amount } pour ce qui doit être recréé en notes privées. L'événement ExternalContractInvoked expose l'adresse cible et le sélecteur, mais pas le calldata.

Packages de référence : Ekubo swap, Vesu lending.

Cette section corrige la fiche technique du brief là où le code dit autre chose. C'est plus important que de la confirmer.

ÉLÉMENT	BRIEF PDF	CODE	VERDICT
Hash Merkle / Fiat-Shamir	Blake3-256	SHA-256	à corriger
Requêtes	32	27	à corriger
Grinding	aucun	16 bits	à corriger
Folding FRI	8	2	à corriger
Soundness	128 bits	124 bits (27q) / 104 bits (22q)	à corriger
Taille de preuve	60-120 KB	80 KB (C0) à 144 KB (C4)	à élargir
Vérification ~1,1M CU	—	aucune mesure committée	à nuancer
Blowup	16	16	OK
Corps Goldilocks	$2^{64}-2^{32}+1$	confirmé des deux côtés	OK
Trusted setup	aucun	aucun, grep exhaustif	OK
Deux phases FRI puis DEEP-ALI	oui	oui	OK
145 chunks pour un shield	oui	145 tx de 1000 B	OK

Pourquoi les paramètres du brief sont ceux d'un fichier mort

stark/src/prover.rs:45-54 construit bien un `ProofOptions::new(32, 16, 0, FieldExtension::None, 8, 31)` avec un commentaire « 128-bit security ». Mais **rien dans le pipeline on-chain ne lit ce fichier**. Le format réellement vérifié sur Solana est un protocole écrit à la main dans stark/src/compact.rs, qui n'importe de winterfell que `BaseElement` et `FieldElement`. Le programme on-chain n'a **aucune** dépendance winterfell.

```
const BLOWUP: usize = 16;
// Soundness: NUM_QUERIES * log2(BLOWUP) + grinding ≈ bits of security.
// 27 * 4 + 16 = 124 bits > 100-bit target. Classical soundness.
const NUM_QUERIES: usize = 27;
const GRINDING_BITS: u32 = 16;
```

stark/src/compact.rs:29-38 — C3, C5 et C6 utilisent 22 requêtes, soit 104 bits

Pourquoi SHA-256 et pas Blake3

Documenté dans stark/Cargo.toml : le syscall `sol_blake3` est derrière un feature runtime inactif sur devnet et mainnet, donc un Blake3 logiciel coûtait ~15k CU par hash et faisait exploser le budget. `sol_sha256` coûte ~85 CU par appel. Blake3 n'apparaît plus que dans le chemin legacy mort. Trois commentaires périmés dans compact.rs et merkle.rs disent encore Blake3 alors que le code appelle sha256.

Nuance à assumer si on te pousse

La formule `queries * log2(blowup) + grinding` est l'heuristique naïve, sans décote de proximity gaps. Et le point OOD `z` est échantillonné dans le **corps de base** ($\sim 2^{64}$), pas dans une extension : le terme Schwartz-Zippel est donc bien en dessous de 124 bits. Le code ne distingue nulle part soundness conjecturée et prouvée. Eux le font explicitement (« 96 bits of **conjectured** soundness »).

Source : compact_proof.rs:50-166, la structure CircuitConfig que le parseur on-chain utilise comme autorité.

ID	NOM	LARG.	LONG.	LDE	PROF.	REQ.	CONTR.	TAILLE PREUVE
0	subscriber_ownership	3	32	512	9	27	3	79 993 B
1	denominated_pool	3	128	2048	11	27	4	120 665 B
2	balance_proof	4	128	2048	11	27	7	121 113 B
3	merkle_path	6	512	8192	13	22	11	138 293 B
4	confidential_balance	4	256	4096	12	27	10	144 041 B
5	transfer	7	512	8192	13	22	28	138 661 B
6	merkle_update	10	512	8192	13	22	19	139 765 B

Correction trouvée dans le code lui-même

lib.rs:229 dit « 10-23 contraintes selon le circuit », et le test dit « C5, width=6, 23 contraintes ». Les deux sont faux. C5 fait **28 contraintes et une largeur de 7** — la colonne d'accumulateur de conservation de valeur a été ajoutée. Les tableaux RLC du vérificateur le confirment : [FeLt::ZER0; 28].

Ce que chaque circuit prouve

ID	ÉNONCÉ
C0	Poseidon(secret) == commitment
C1	nullifier = Poseidon(np, secret) ; epoch_hash = Poseidon(epoch, mint) ; commitment = Poseidon(nullifier, epoch_hash)
C2	chaîne de 4 Poseidon pour un commitment de balance. Le range check n'est PAS dans l'AIR , il est fait on-chain
C3	inclusion Merkle par Poseidon hash2 par niveau, profondeur canonique 15
C4	mise à jour de balance confidentielle, 7 Poseidon chaînés. Conservation appliquée on-chain, pas dans l'AIR
C5	transfert shielded 2-in-2-out, 14 Poseidon chaînés + colonne d'accumulateur de conservation
C6	remplacement de feuille old_root → new_root avec témoins de chemin partagés

Compute units : ce qu'on peut réellement affirmer

Il n'y a **aucune mesure de CU committée** dans le dépôt. Les seules assertions sont expect(cu).to.be.lessThan(1_400_000) sur les deux phases de C5, avec les valeurs réelles imprimées à l'exécution et jamais enregistrées. Tout chiffre du type « 1,1M CU » vient d'une note de mémoire, pas du code.

Ce qui est solidement démontrable en revanche : deux signatures devnet sont citées **dans le code** comme ayant consommé le budget complet de 1,4M à l'étape 4, ce qui a forcé le découpage en deux phases.

Formulation sûre : « les deux phases passent sous le plafond de 1,4M, c'est asserté en test ; le chiffre exact n'est pas figé dans le dépôt et je ne vais pas l'inventer ».

Nomenclature à corriger

Le dépôt appelle partout 1,4M le « plafond par instruction ». Sur Solana, 1 400 000 est MAX_COMPUTE_UNIT_LIMIT, un plafond **par transaction**. Le découpage marche parce qu'il met les deux appels lourds dans **deux transactions séparées**, et c'est exactement ce que fait le code mobile.

Le pool

Seed PDA	denominated_pool_v4
Profondeur d'arbre	$15 - 2^{15} = 32\,768$ notes par pool, doit éгалer CANONICAL_DEPTH du circuit C6
Racines historiques	100
Dénominations	SOL : 0.1, 1, 10, 100, 500, 1000 · USDC : 1, 10, 100, 1000, 10k, 20k, 50k
Commitment	Poseidon(nullifier, epoch_hash), avec nullifier = Poseidon(np, secret) et epoch_hash = Poseidon(epoch, mint)
Hash de l'arbre	Poseidon Goldilocks t=3, 30 rounds pleins, S-box x^7 , MDS circulante [[3,1,1],[1,3,1],[1,1,3]] — différent du SHA-256 des commitments de la preuve, distinction correcte et voulue
Vérificateur	DGY37k3Jt7cbrfNa9rxyLZVcFB7S7A2NqtVpkh9fWQvs

La correction de pitch la plus importante

« **Aucune courbe elliptique dans la découverte du destinataire** » est faux. Une multiplication scalaire X25519 s'exécute des deux côtés de chaque opération de découverte.

```
export function deriveSharedSecret(privateKey, publicKey) {
  return nacl.scalarMult(privateKey, publicKey); // X25519
}
```

utils/crypto.ts:41-46 — appelée en derive.ts:49-52, 142, 195 et scan.ts:124

Ce qui est vrai et défendable, c'est que le combineur est **OR-sûr** : la confidentialité de la découverte survit à une casse totale de X25519. Le code le dit : « Security holds if EITHER the classical or post-quantum scheme is secure. »

Formulation de remplacement : la découverte du destinataire est résistante au quantique. Le secret partagé ne peut pas être reconstitué sans la clé secrète ML-KEM-768, même par un adversaire qui casse X25519.

Le protocole exact, côté émetteur

```
r      = keypair X25519 éphémère
s_c    = X25519(r, V)
(ct, s_q) = ML-KEM-768.Encaps(K)
s      = HKDF-SHA256(ikm = s_c || s_q, info = "p01-hybrid-stealth-v2" || SHA256(R || ct), L=32)
viewTag = SHA256(s)[0]
seed    = HKDF-SHA256(ikm = s, salt = spendingPubKey, info = "p01-stealth-seed", L=32)
stealth = Ed25519_keygen_from_seed(seed)
```

Ce qui te distingue d'ERC-5564 et de Monero

Il n'y a **aucune addition de points de courbe** dans la dérivation. L'adresse furtive est une clé Ed25519 fraîche issue d'un seed HKDF ; la clé de dépense publique du destinataire ne sert que de sel HKDF. Conséquence à assumer : la clé de dépense n'est pas requise pour dépenser — clé de vue + secret KEM suffisent à reconstruire la clé privée.

Primitives	ML-KEM-768 de @noble/post-quantum ^0.6.1 — pk 1184 B, sk 2400 B, ct 1088 B, secret 32 B
Meta-adresse v2	1249 octets bruts en base58, ~1700 caractères — contrainte UX réelle, pas de forme courte dans le code
Transport	PDA StealthAccountV2, 1220 B, KEM_CHUNK_SIZE = 544 donc 1088 = exactement 2 chunks, soit 3 transactions par envoi
Programme	FgKhXakZGsd4PdIGgACyY8gwj1JLMYA691yQr2PhUNfL
SDK	@protocol-01/specter-sdk@0.4.1, npm public, publié le 17/07/2026 à 12:17 UTC, 3 min après le commit du transport chunké

Les dire avant qu'on te les demande, c'est ce qui crédibilise le plus. Liste réelle, établie sur le code.

Le view tag ne filtre rien

checkViewTag calcule le tag depuis le secret **hybride**, donc il exécute une décapsulation ML-KEM-768 complète plus un scalarmult X25519 **avant** de pouvoir rejeter.

Le commentaire on-chain qui dit que le tag « évite la décapsulation coûteuse » (state/stealth.rs:134-137, avec un taux de rejet annoncé de 99,6 %) contredit directement l'implémentation cliente scan.ts:124-138.

Travail fait deux fois

Après le passage du view tag, verifyStealthOwnership refait à l'identique le scalarmult, la décapsulation et le HKDF. Rien n'est mémorisé.

Bande passante non bornée

fetchAnnouncements ignore son intervalle de slots (paramètres _fromSlot, _toSlot) et getProgramAccounts télécharge **toutes** les annonces de 1220 B jamais créées, puis tronque côté client. Pas de dataSlice, pas de pagination.

Pas de flag de complétude

init_stealth_v2 remplit le ciphertext de zéros et rien n'impose que les deux chunks soient écrits. Un PDA à moitié écrit est indistinguable pour le scanner.

3 transactions non atomiques

tx1 crée le PDA **et** transfère les fonds, avant l'écriture des chunks. Si un chunk échoue, l'argent est à une adresse furtive dont le ciphertext est incomplet, donc introuvable par le destinataire.

Émetteur identifiable

Le PDA est dérivé de la vraie pubkey de l'émetteur (seeds = ["stealth_v2", sender, stealthAddress]), donc chaque annonce lui est publiquement attribuable, indépendamment du signataire de la transaction.

Dépense non PQ

L'adresse furtive est Ed25519, donc dépensable par une signature classique. La découverte est PQ, la dépense ne l'est pas. Le SDK le documente honnêtement et livre un module WOTS+, mais le vérificateur on-chain qui imposerait la seconde signature **n'existe pas**.

Le reste, en bref

- announcement-v2.ts a zéro test et n'est pas exporté de l'API publique
- Le chemin SPL-token est mort en v2 : sendSingleTransfer lève dès qu'un tokenMint est fourni. Toute la couche PQ est **native-SOL uniquement**
- constants.ts annonce un program id mainnet-beta qui n'est pas le declare_id du programme actuel
- Les discriminateurs Anchor sont recalculés côté client depuis sha256('global:<name>') plutôt que lus d'un IDL — casse silencieuse si un nom d'instruction change
- Aucun test de parité entre boundary_assertions_for_circuit (prover) et get_boundary_assertions (vérificateur), contrairement aux constantes périodiques qui, elles, sont couvertes par 7 tests
- Marge de **959 octets seulement** entre C4 (144 041 B) et UNIFORM_PROOF_SIZE (145 000 B)

3.1 Système de preuve et corps

	STRK20	PROTOCOL 01
Système	Stwo, Circle STARK	Winterfell 0.10.3 comme lib de corps + protocole compact écrit main
Corps	M31, $p = 2^{31}-1$, tour CM31/QM31	Goldilocks, $p = 2^{64}-2^{32}+1$, pas d'extension
Astuce structurelle	courbe circulaire, ordre $p+1 = 2^{31}$	sous-groupe lisse natif
Hash de commitment	Blake2s-256 (défaut), Poseidon252 (option L1)	SHA-256 (contrainte de syscall Solana)
Merkle	« lifté » (nœud = enfants + colonnes)	standard
Blowup	2	16
Requêtes	70	27 ou 22
Grinding	26 bits + PoW d'interaction	16 bits
Soundness annoncée	96 bits conjecturés	124 / 104 bits, classique, formule naïve
Point OOD	QM31 (extension quartique)	corps de base $\sim 2^{64}$
Trusted setup	aucun	aucun
Zero-knowledge	pas ZK par défaut	pas de masquage ZK non plus

Honnêteté dans les deux sens

Eux ont un blowup de 2 avec 70 requêtes, ce qui n'atteint 96 bits que sous l'hypothèse conjecturale de list-decoding. Toi tu as un blowup de 16, plus conservateur par requête, mais tu échantillones le point OOD dans le corps de base au lieu d'une extension, ce qui plafonne le terme Schwartz-Zippel bien plus bas que ta figure annoncée. **Aucun des deux ne devrait clamer un chiffre nu sans dire quel régime.**

3.2 Modèle d'écriture des contraintes

C'est la différence structurante. Eux : une AIR de CPU fixe, le programme est de la donnée, écrire une fonctionnalité privacy = écrire du Cairo. Toi : une AIR par circuit, écrite à la main. **Et le brief sous-estime le coût.**

Changer UNE contrainte de transition = 4 sites manuels + 1 table générée

1. l'AIR — `evaluate_<circuit>_transition`, plus la liste `TransitionConstraintDegree` et les colonnes périodiques
2. le quotient prover — `compute_quotient_lde_circuit_N` dans `compact.rs`
3. l'évaluateur OOD du vérificateur — `evaluate_transition_at_ood_circuit_N`
4. **la re-vérification par requête** — `verify_constraints_<circuit>`, copie manuelle séparée de la même logique
5. les coefficients périodiques `CN_*_COEFS` dans `periodic_consts.rs` (666 KB générés)

Une contrainte de **boundary** ajoute encore deux sites miroir, **sans aucun test de cohérence croisée**. Le seul garde-fou automatisé existant porte sur les constantes périodiques : 7 tests `circuit_N_periodic_coeffs_match_verifier_constants`.

Formulation juste devant un ingénieur StarkWare : chez moi une contrainte vit dans quatre implémentations manuelles synchronisées, dont deux ne sont couvertes par aucun test de parité. C'est ma dette principale, et c'est exactement ce que le modèle Cairo supprime.

3.3 Lieu et coût de vérification

	STRK20	PROTOCOL 01
Vérificateur	S-two en Rust dans le gateway / séquenceur	programme Solana on-chain
Plafond de calcul	aucun ; contrainte économique (125 L2gas/octet + 5 de stockage + 10M de buffer)	1,4M CU par transaction, dur
Découpage	aucun	deux transactions (phase 1 puis DEEP-ALI)
Sécurité héritée	dégradée : vérifié par le consensus, pas par le SNOS	Solana pleine, tous les validateurs
Rôle du contrat	lit seulement proof_facts via syscall	parse la preuve entière

L'arbitrage, et pourquoi tu n'es pas en position de faiblesse

Eux n'ont pas de plafond de calcul **parce qu'ils ont sorti la vérification de la couche contrat**, et StarkWare écrit que ça dégrade la sécurité en phase 1. Toi tu paies 1,4M CU **parce que ta preuve est vérifiée par la chaîne elle-même**, sans confiance additionnelle.

À dire tel quel si on te compare : chez eux la preuve est vérifiée par le séquenceur, pas par le contrat, et ils documentent que la sécurité est dégradée en phase 1 en attendant la récursion dans le SNOS. Chez moi elle est vérifiée on-chain par tous les validateurs, et je paie ça en compute units. Ce sont deux positions différentes sur le même arbitrage, pas une supériorité.

3.4 Transport de la preuve

	STRK20	PROTOCOL 01
Taille	non publiée pour STRK20 ; S-two 808 KB à 1315 KB ; modèle de prix SNIP-36 sur 500 KB	80 KB à 144 KB, paddée à 145 000 B uniformes
Chemin	champs proof / proof_facts d'Invoke V3, hors calldata	145 tx de chunk de 1000 B dans un PDA, après 14 tx de resize de 10 240 B
Persistance	pas dans les blocs, pas au feeder gateway, gardée quelques semaines	PDA, rent immobilisé, fermé après usage
Rent	aucun	~0,87 SOL de float transitoire

Un point que le brief n'exploite pas

Ta preuve est 5 à 10 fois plus petite que la leur. Goldilocks avec blowup 16 et 22-27 requêtes produit une preuve nettement plus compacte que M31 avec blowup 2 et 70 requêtes. Leur coût de transport est absorbé par le protocole, le tien par 159 transactions. C'est un vrai avantage technique, à condition de dire les deux moitiés.

Pourquoi 145 000 octets uniformes

UNIFORM_PROOF_SIZE est défini côté **client** dans le service STARK mobile, pas en Rust. Il ferme deux fuites : L14 (fingerprint par taille de preuve) et, couplé au PDA à nonce d'init_proof_buffer_v2, L13 (circuit_id). Marge réelle au-dessus de C4 : 959 octets. C'est tendu, et il faut le savoir avant qu'on te le demande.

3.5 Modèle de note

	STRK20	PROTOCOL 01
Structure	{ packed_value: felt252, token: ContractAddress }	commitment Poseidon
Identité	adresse de stockage	feuille dans un arbre de Merkle
Montant	masqué par hash-and-add, salt 120 bits	la dénomination du pool, pas une entrée de circuit
Anonymat	pseudo-aléatoire des slots	ensemble d'anonymat de l'arbre
Multi-actifs	un pool pour tous les ERC-20	un pool par (token, dénomination) — 6 SOL + 7 USDC
Conservation	registre de balance temporaire on-chain	dénomination + accumulateur C5

Le cœur de la conversation technique

Tes pools dénommés sont sûrs par construction sur la valeur, mais fragmentent l'ensemble d'anonymat en 13 pools. Eux ont un ensemble unifié, mais **remplacent l'anonymat d'ensemble par du pseudo-aléatoire d'adresse** : la confidentialité tient tant que le channel_key reste secret, et la clé de vue qui le régénère est chiffrée on-chain avec de l'ECDH sur courbe.

C'est exactement là que ton argument PQ mord, et c'est plus fort que sur un modèle à Merkle : chez eux, casser la courbe ne révèle pas seulement qui a reçu, ça révèle où sont les notes, parce que l'adressage lui-même dérive du secret.

3.6 Accumulateur

STRK20

Aucun. L'équivalent fonctionnel de ta fenêtre de racines historiques est la fenêtre de validité de preuve (450 blocs, ~15 min) ancrée à un bloc de base, plus la maturation de 10 blocs.

PROTOCOL 01

Arbre de Merkle Poseidon-Goldilocks de profondeur 15 (32 768 notes par pool), 100 racines historiques, maintien des sous-arbres on-chain via C6.

Ce n'est pas le même mécanisme : le tien résout la concurrence sur l'état de l'arbre, le leur la fraîcheur de l'exécution virtuelle.

3.7 Nullifiers

	STRK20	PROTOCOL 01
Formule	Poseidon(NULLIFIER_TAG, channel_key, token, index, 0, owner_private_key)	Poseidon(nullifier_preimage, secret)
Stockage	Map<felt252, bool> WriteOnce	PDA
Propriété notable	l'émetteur ne peut pas le calculer, donc ne peut pas surveiller la dépense	—

Un point où ils sont devant, à concéder

Leur asymétrie émetteur / destinataire est élégante et tu ne l'as pas. Le concéder spontanément vaut mieux que de se le faire pointer.

L'axe central. C'est là que se joue le projet, et le résultat est bidirectionnel.

	STRK20	PROTOCOL 01 (SPECTER)
Mécanisme	seau par destinataire, indexé par adresse Starknet en clair	adresse furtive dérivée, annonce publique
Pré-requis	destinataire enregistré on-chain avec clé de vue publique	aucun — le destinataire n'a rien à faire avant
Coût de scan	proportionnel à ta propre activité	proportionnel au volume total du pool
Filtre rapide	pas nécessaire (seau déjà filtré)	view tag 1 octet, mais il ne filtre rien
Crypto	ECDH courbe STARK + Poseidon	hybride X25519 + ML-KEM-768, OR-sûr
Résistance quantique	non	oui pour la découverte, non pour la dépense
Fuite	adresse du destinataire visible à l'ouverture de canal	émetteur identifiable via les seeds du PDA
Ciphertext	felt252 dans le stockage	1088 B en 2 chunks dans un PDA

Lecture honnête

Ils gagnent sur la scalabilité du scan. Tu gagnes sur la résistance quantique et sur le fait qu'un destinataire non enregistré peut recevoir.

Leur modèle a un défaut structurel que le tien n'a pas : il faut connaître et publier l'adresse du destinataire, et l'ouverture de canal la révèle. Le tien a un défaut qu'ils n'ont pas : tu télécharges tout le pool à chaque scan.

Le vrai insight du croisement

Ton architecture d'adresse furtive résout leur problème de liaison à l'ouverture de canal, et leur architecture de seau résout ton problème de coût de scan.

C'est la conversation la plus intéressante que tu puisses avoir aujourd'hui, et elle est bidirectionnelle, pas vendeuse. Un ingénieur StarkWare qui entend ça comprend immédiatement que tu as lu leur code, et la discussion change de registre.

Chiffres de scan, côté Protocol 01

Par annonce : 1 scalarmult X25519 + 1 décapsulation ML-KEM-768 + 2 SHA-256 (doublés à 2 + 2 sur un hit de tag), plus 1220 octets de téléchargement RPC sans pagination. **Aucun benchmark n'existe dans le dépôt** : le coût wall-clock pour 1000 annonces sur téléphone n'est pas mesuré. Ne pas avancer de chiffre.

3.9 Chiffrement des notes

	STRK20	PROTOCOL 01
Schéma	ECDH courbe STARK, KDF Poseidon, masque additif	X25519 + ML-KEM-768, HKDF-SHA256
Extensible	non — figé dans le contrat, aucun champ de version	oui côté SDK
Versionnage	suffixe :V1 dans 17 tags Poseidon	version explicite dans le format d'annonce (1 octet)
Migration	non documentée ; clés de vue immuables (WriteOnce)	v1 rejeté par défaut, opt-in pour scan historique

3.10 Viewing keys et compliance

	STRK20	PROTOCOL 01
Existence	native, obligatoire pour utiliser le pool	aucune
Chiffrement de la clé	ECDH éphémère courbe STARK + masque Poseidon, on-chain, immuable	—
Auditeur	une seule clé publique ; « seuil » en doc, pas dans le contrat	—
Portée	historique bidirectionnel complet, traçage avant et arrière	—
Screening	on-chain, signature SNIP-12, non contournable	aucun

3.11 Modèle de compte

	STRK20	PROTOCOL 01
Base	abstraction de compte native ; is_valid_signature réutilisé pour la dépense	Ed25519 / PDA Solana
Bénéfice	multisig, hardware, smart accounts sans changement de protocole	—
Coût	compte déployé et finalisé ~10 blocs avant de pouvoir s'enregistrer	—
Adresse dérivée	ne peut pas recevoir sans enregistrement préalable	adresse furtive reçoit directement
Chemin PQ signature	AA permet déjà un signer PQ — Falcon-512 démontré à ~9,5M L2 gas	WOTS+ côté client, vérificateur on-chain absent

Point d'honnêteté sur les signatures PQ

Ne pas dire « on ne se disperse pas parce que Falcon est déjà sur leur roadmap ». Leur AA permet déjà un wallet Falcon-512 déployé sur mainnet par un tiers, à ~9,5M L2 gas. **Sur cet axe ils sont devant toi.** Ta valeur est sur le KEM — et c'est vrai précisément parce qu'ils ont bien couvert la signature et pas du tout le KEM.

3.12 Composabilité et maturité

	STRK20	PROTOCOL 01
DeFi	swaps privés AVNU + Ekubo live ; anonymizers	à construire
Wallets	Ready live, Xverse en cours	app mobile maison
Statut	mainnet depuis avril-mai 2026	devnet uniquement
Audit	OpenZeppelin + ~60 fichiers Lean 4	audit de soundness interne
Open source	Apache 2.0 depuis le 8 juillet 2026	SDK npm public, monorepo privé

Deux formulations à ne pas employer

Le brief dit déjà de ne pas dire « STRK20 n'est pas post-quantique ». La vérification adversariale en a trouvé une **deuxième**, également fautive et qui te ferait corriger sur place : **ne dis pas non plus « la confidentialité est hors périmètre »**. STRK20 est leur produit de confidentialité, livré deux mois avant la roadmap.

Formulation vérifiée, à utiliser telle quelle

La roadmap post-quantique de StarkWare du 30 juin couvre le hachage et les commitments : Pedersen remplacé par BLAKE2 sur le state commitment, la dérivation d'adresse de contrat et le hash de config de l'OS, plus la migration des contrats existants et les dépendances Ethereum. Elle ne dit pas comment la couche de chiffrement qu'ils ont déjà livrée dans STRK20 sera migrée : les balances chiffrées, et la clé de vue chiffrée on-chain qu'un auditeur désigné peut déchiffrer. C'est une asymétrie qui mérite une question, pas une accusation. Le système de preuve est basé sur des hashes et tient. La couche de confidentialité a été livrée deux mois avant la roadmap et n'y figure pas.

Les 3 phases

1 — migrations de hash (state commitment, dérivation d'adresse, config OS) Pedersen→BLAKE2, ~2 mois. 2 — stockage des contrats existants. 3 — dépendances Ethereum (secp256k1/r1, blobs KZG), bloquées sur Ethereum

Falcon-512

cité **pour le consensus seulement**, comme exemple non contraignant (« such as »). Pas un engagement sur la signature des comptes. **Aucun SNIP** pour Falcon ni les signatures PQ de compte — vérifié de façon adversariale

Comptes

couverts indirectement : « Starknet users can deploy a PQ wallet today », sans changement de protocole

Les deux cibles concrètes

CIBLE A — LA PLUS FORTE

La séquestration de clé de vue (EncPrivateKey)

```
enc_private_key = h(ENC_PRIVATE_KEY_TAG, rK.x) + private_key
```

- ça protège l'historique bidirectionnel entier d'un utilisateur, pas une transaction
- permanent et immuable par conception : l'exposition ne décroît jamais
- une seule clé d'auditeur cassée expose **tous les utilisateurs à la fois**, et désanonymise rétroactivement tout le pool depuis mai 2026

CIBLE B

L'ouverture de canal (EncChannelInfo)

```
enc_channel_key = h(CHANNEL_KEY_TAG, rK.x) + channel_key
enc_sender_addr = h(SENDER_ADDR_TAG, rK.x) + sender_addr
```

Chez eux, l'adressage des notes dérive du secret de canal. Casser la courbe ne révèle pas seulement qui a reçu combien, ça révèle où toutes les notes sont stockées. La confidentialité et l'adressage reposent sur la même clé, et cette clé est chiffrée on-chain avec de l'ECDH sur courbe elliptique, pour toujours.

Ce qui les protège déjà, à reconnaître d'emblée

Stwo est basé sur des hashes et survit à Shor — à dire en premier. Blake2s est déjà partiellement déployé, l'AA permet un signer PQ dès aujourd'hui. Et surtout : un chiffrement additif dans un corps premier n'a pas de structure exploitable par Shor **en lui-même** ; la vulnérabilité est entièrement dans l'ECDH qui produit $rK.x$. C'est le seul composant qu'un KEM remplace. **Le changement est chirurgical** — c'est ton meilleur argument.

Ce qui tombe

L'architecture cible de la roadmap (émetteur dérive une adresse furtive, STRK20 transfère dessus) ne peut pas être implémentée. Ne pas essayer de la forcer. Ne pas proposer de reconstruire un système de notes non plus — le piège n°4 du brief tient toujours.

Ce qui devient le projet

Remplacer l'ECDH sur courbe STARK par un KEM hybride dans les deux structures chiffrées de STRK20, sans toucher au reste du protocole.

Ce qui rend ça défendable en 12 heures :

- Le périmètre est **exactement deux fonctions**. `_compute_shared_x` produit `rK.x`, et tout le reste du système consomme un `felt252` de matière clé. Si `rK.x` est remplacé par un secret issu d'une décapsulation ML-KEM réduit dans le corps, **rien d'autre ne change** : ni la structure des notes, ni les nullifiers, ni la discovery, ni les phases.
- Le combineur OR-sûr de Specter existe déjà et est testé : `HKDF(classic || pq, info = transcript)`. Il se transpose directement, en substituant Poseidon à SHA-256 pour rester dans leur domaine de hash.
- L'argument « on ne fait pas de mixer maison, on se branche au-dessus » devient **littéralement vrai au niveau de la ligne de code**.

Détail qui compte : stockage, pas events

STRK20 ne met **aucun** payload chiffré dans les events — toute la discovery lit du stockage. Un `pq_announcer` qui émettrait des events divergerait de leur architecture et se ferait remarquer. Packing : 1088 octets à 31 octets par `felt` = **36 felts**. En stockage plutôt qu'en events, ce qui évite aussi le coût de 5 120 L2gas par `felt` de données d'event (184 320 L2 gas, soit 4,6 L1-gas-équivalent : pas prohibitif, mais inutile).

Effort de portage, mesuré fichier par fichier

MODULE	WEB3	PUBLICKEY	CONNECTION	VERDICT
<code>quantum/*</code> (4 fichiers)	0	0	0	déjà portable
<code>stealth/index.ts</code>	0	0	0	déjà portable
<code>stealth/quantum.ts</code>	1	5	0	~5 lignes
<code>stealth/derive.ts</code>	1	6	0	cosmétique
<code>stealth/generate.ts</code>	1	8	0	cosmétique
<code>stealth/scan.ts</code>	1	12	6	à remplacer par un adaptateur
<code>stealth/announcement-v2.ts</code>	1	20	2	à remplacer par un adaptateur

Bonne nouvelle par rapport à l'estimation de la roadmap (117 références sur 8 fichiers, 3–5 h de refactor). La réalité : `PublicKey` est un wrapper de 32 octets et `Keypair` ne sert qu'à `.PublicKey.toBytes()`. Le vrai travail est concentré sur **deux fichiers**, et ce sont précisément les deux qui doivent être réécrits de toute façon. Et pour la cible retenue, tu n'as même pas besoin de `stealth/` : il te faut `quantum/` et `kemEncapsulate / kemDecapsulate / deriveHybridSharedSecret`, qui n'ont aucune dépendance chaîne.

Règle de coupe révisée

MVP défendable : un handshake ML-KEM-768 dont le secret partagé est réduit dans le corps STARK, utilisé pour masquer un `channel_key` selon leur formule exacte $h(\text{TAG}, \text{secret}) + \text{value}$, avec le ciphertext stocké dans un contrat Cairo sur Sepolia, et une démonstration que l'émetteur et le destinataire dérivent le même `channel_key`.

Ça prouve la substituabilité au niveau exact où elle compte, **sans dépendre du proving service, sans dépendre du screening de dépôt, sans dépendre de l'accès au pool** — les trois choses qui peuvent te bloquer et dont aucune n'est sous ton contrôle.

Devenues inutiles — déjà répondues par le code

- Q1 adressage du destinataire → seau par adresse en clair, pas de tag, scan de son propre seau
- Q2 chiffrement extensible → **non**, figé dans le contrat, aucun champ de version
- Q3 schéma des viewing keys → ECDH éphémère courbe STARK + masque Poseidon, immuable. **Garde-la, mais pose-la en affirmant : ça montre que tu as lu leur code**
- Q5 transfert vers adresse dérivée → **non**, destinataire enregistré obligatoire
- Q6 SDK gated ou public → open source Apache 2.0
- Q8 coût des events Cairo → 5 120 L2gas/felt de données, mais STRK20 n'utilise pas d'events pour les payloads

Les vraies bloquantes — première heure

1. **URL du proving service Sepolia.** Aucune valeur publique nulle part ; tous les exemples utilisent `process.env.PROVING_SERVICE_URL` sans défaut, et la demo tombe sur un mock si non défini. Endpoint hébergé par StarkWare ? Credentials ? Rate limits ?
2. **URL de l'indexer de discovery.** Même problème (`process.env.INDEXER_URL`). Endpoint hébergé, ou faut-il faire tourner le container ?
3. **Adresse du pool sur Sepolia.** Le mainnet est publié, Sepolia est confirmé supporté mais l'adresse n'est publiée nulle part — elle est résolue par le wallet à l'exécution.
4. **Accès npm au SDK.** `@starkware-libs/starknet-privacy-sdk` renvoie 404 sur le registre public ; c'est sur GitHub Packages. Token nécessaire, ou install git ?

Les techniques qui valent le détour

5. **Migration du format chiffré.** Aucun champ de version sur Note ni EncChannelInfo, et clés de vue WriteOnce. Si vous passez le chiffrement en post-quantique, comment migrent les canaux existants et les clés déjà séquestrées ? **La meilleure question du lot** : concrète, issue du code, sans réponse documentée.
6. **Qui porte la couche chiffrement dans la roadmap PQ ?** Oubli, séquencement, ou chantier ouvert non publié ?
7. **La clé de vue privée en clair vers le prover.** Modèle de confiance, attestation TEE prévue, politique de rétention ? À poser sans agressivité : c'est un choix d'architecture explicite en phase 1, pas une négligence.
8. **Auditeur à seuil.** Les docs le disent supporté, le contrat stocke un seul `felt252`. Déployé aujourd'hui ? Porteurs de parts ? Process de requête ?
9. **FPI vs Elliptic.** Qui signe, qui fournit la donnée, qui contrôle `blockOverrideAddresses` ?
10. **Le r éphémère réutilisable.** Corrigé dans la classe déployée, ou assumé côté client ?

Entre pairs

11. **M31 contre Goldilocks.** Quelles contraintes le choix du corps met-il sur les décompositions en bits et les range checks dans leurs circuits privacy ? Tu as un vrai vécu : tu as différé les range checks sur `confidential_balance` et `transfer` précisément parce que la décomposition en bits faisait exploser le budget CU.
12. **Circuits sous-contraints en Cairo.** Avec une AIR de CPU fixe, le risque se déplace du circuit vers la logique Cairo. Ils ont ~60 fichiers Lean 4 : demande ce que Lean couvre et ce qu'il ne couvre pas.
13. **SNIP-36 phase 2.** Récursion dans le SNOS et vrai ZK différés sans calendrier. Quel horizon, qu'est-ce qui débloque ?



À NE PLUS DIRE	À DIRE
« Aucune courbe elliptique dans la découverte du destinataire »	« La découverte est résistante au quantique : le secret ne se reconstitue pas sans la clé ML-KEM-768, même si X25519 tombe »
« STRK20 n'est pas post-quantique »	« Le système de preuve tient. La couche de chiffrement livrée en avril n'est pas dans la roadmap PQ de juin »
« La confidentialité est hors périmètre de leur roadmap »	« La roadmap est silencieuse sur la migration PQ d'une couche de chiffrement qu'ils ont déjà livrée »
« Blake3-256, 32 requêtes, folding 8, pas de grinding, 128 bits »	« SHA-256, 27 ou 22 requêtes, folding 2, grinding 16 bits, 124 ou 104 bits en soundness classique »
« 1,1M CU sur 1,4M »	« Les deux phases passent sous 1,4M, c'est asserté en test ; le chiffre exact n'est pas committé »
« Preuve de 60 à 120 KB »	« 80 à 144 KB selon le circuit, paddée à 145 000 octets pour ne pas fuiter le circuit »
« Plafond de 1,4M CU par instruction »	« par transaction — c'est pour ça que le split est en deux transactions »
« ANSSI recommande ML-KEM plus FrodoKEM »	Retirer entièrement. L'hybridation ANSSI est PQ + classique, pas PQ + PQ. Les deux sont des alternatives listées séparément, et ANSSI dit ne pas publier de liste fermée d'algorithmes recommandés
« Les régulateurs visent 2030 »	Vrai mais à sourcer : ANSSI phase 3 « probablement pas avant 2030 » ; UE, trois jalons 31/12/2026, 31/12/2030, 31/12/2035
« Le SDK est gated »	« Le SDK est open source Apache 2.0 depuis le 8 juillet. Ce qui est gated, c'est le proving service et l'indexer »

Sources

STRK20 / Starknet, primaires

- github.com/starkware-libs/starknet-privacy — contrat Cairo, discovery Rust, SDK TS, preuves Lean. Apache 2.0
- community.starknet.io — SNIP-36, vérification in-protocol
- starkware.co/blog — roadmap PQ du 30 juin 2026
- github.com/starkware-libs/stwo et stwo-cairo
- ePrint 2024/278 (Circle STARKs), ePrint 2021/1063 (Cairo)
- docs.starknet.io — fees, S-two book
- @starknet-io/types-js@0.10.3 — Wallet API v0.10.3
- openzeppelin.com/news/privacy-contracts-audit
- strk20-by-example.org — communautaire, pas officiel, mais cite le source Cairo verbatim

Protocol 01, internes

- stark/src/compact.rs — protocole réellement vérifié
- stark/src/prover.rs — chemin legacy mort
- programs/p01_stark_verifier/src/*
- programs/zk_shielded/src/state/*
- packages/specter-sdk/src/*, programs/specter/src/
- apps/mobile/services/stark/index.ts
- apps/mobile/services/denominatedPool/index.ts

Réserves de vérification — à lever soi-même

- L'absence de termes liés au chiffrement dans la roadmap PQ a été établie via un outil qui résume, pas un grep brut. **Ouvrir la page et faire Ctrl-F avant tout usage public** — c'est la vérification manuelle la plus rentable
- Les compute units de Protocol 01 ne sont mesurés nulle part dans le dépôt
- L'affirmation ANSSI ML-KEM+FrodoKEM a été réfutée sur source primaire et doit être retirée
- Le temps de preuve STRK20 a deux valeurs contradictoires publiées (~29 s et ~4 s), non réconciliées par les sources